MODUL PRAKTIKUM PEMROGRAMAN BACKEND LANJUT

Pertemuan 11 — Background Jobs & Asynchronous Processing

EDISI PUZZLE — modul ini memuat kesalahan yang disengaja

Program Studi	D4 Teknik Informatika — Fakultas Vokasi, Universitas Airlangga
Mata Kuliah	Praktikum Pemrograman Backend Lanjut (2 SKS) — Semester 5
Alokasi Waktu	100 menit praktikum terbimbing + 100 menit tugas mandiri

Tujuan Pembelajaran

Setelah menyelesaikan pertemuan ini, mahasiswa diharapkan mampu:

1. Memutuskan task mana yang layak dipindahkan ke background, dengan alasan yang dapat diukur: waktu tanggap, lama lock database dipegang, dan dampak kegagalan.
2. Menjelaskan arsitektur job queue — producer, broker, worker — serta memetakan konsep Asynq ke konsep Fiber yang sudah dikenal.
3. Menjelaskan masalah dual write antara PostgreSQL dan Redis, lalu menyelesaikannya dengan pola transactional outbox.
4. Menjelaskan mengapa queue hanya menjamin at-least-once, lalu merancang handler yang idempoten dengan idempotency key.
5. Membedakan temporary failure dan permanent failure, serta mengendalikan retry, backoff, dan archiving task.
6. Menjalankan API dan worker sebagai proses terpisah dari satu kode sumber, lengkap dengan graceful shutdown.
7. Membuktikan ketahanan sistem dengan menguji kegagalan secara sengaja — mematikan Redis, menggagalkan pengiriman, mengirim task ganda — lalu menemukan dan memperbaiki kesalahan yang ditanam.

> **Mengapa kita membutuhkan background job sekarang**
>
> Sejak Modul 9, mendaftar kelas adalah transaction yang cepat: beberapa milidetik saja. Kampus kemudian meminta satu fitur kecil — setiap perubahan KRS dikirimi email konfirmasi.
>
> Fitur "kecil" itu ternyata mengubah segalanya. Server email butuh satu sampai dua detik untuk menjawab, kadang sibuk, kadang mati. Bagian A menunjukkan dengan angka hasil pengukuran: bila email dikirim di tempat yang salah, 27 dari 30 mahasiswa gagal mendaftar pada detik yang sama — bukan karena kelasnya penuh, tetapi karena menunggu email orang lain.

A. Landasan Teori

A.1 Kasir dan Dapur

Bayangkan sebuah kedai kopi. Kasir menerima pesanan, mencatatnya, memberi nomor antrean, lalu langsung melayani pelanggan berikutnya. Kopinya dibuat barista di belakang. Kasir tidak berdiri menunggu kopi matang sebelum melayani orang berikutnya — kalau begitu, antrean di depan kasir akan mengular walaupun barista menganggur.

Endpoint POST /courses/:id/enroll adalah kasir. Mencatat KRS ke database adalah pekerjaan kasir: wajib, cepat, dan hasilnya harus diketahui mahasiswa saat itu juga. Mengirim email konfirmasi adalah pekerjaan barista: lambat, boleh terlambat beberapa detik, dan mahasiswa tidak perlu menunggunya.

Tabel berikut adalah hasil pengukuran pada aplikasi kita sendiri, dengan server email tiruan yang butuh 1,5 detik per pengiriman:

Di mana email dikirim	1 mahasiswa	30 mahasiswa serentak, kelas yang sama	Bila server email gagal
Di dalam transaction KRS	± 1,5 detik	3 berhasil, 27 gagal (timeout 5 detik habis)	KRS ikut dibatalkan
Setelah commit, masih di handler	± 1,5 detik	30 berhasil, masing-masing ± 1,5 detik	Response 500 walaupun KRS tersimpan
Di background (modul ini)	± 3 milidetik	30 berhasil, paling lama 0,11 detik	Di-retry otomatis; KRS tidak terpengaruh

Dua baris pertama layak direnungkan:

- **Email di dalam transaction.** Ingat Modul 9: IncrementEnrolled me-lock row kelas sampai transaction selesai. Bila transaction itu juga menunggu email 1,5 detik, mahasiswa kedua menunggu 1,5 detik, mahasiswa ketiga 3 detik, dan seterusnya. Dalam 5 detik hanya 3 orang yang sempat dilayani. Lock database dipegang selama menunggu sesuatu yang sama sekali bukan urusan database.
- **Email setelah commit.** Lock sudah dilepas sehingga tidak ada yang menunggu giliran di database, tetapi setiap mahasiswa tetap menunggu 1,5 detik. Lebih buruk lagi: saat server email gagal, mahasiswa menerima 500. Ia mencoba lagi — dan mendapat 409 ALREADY_ENROLLED. Dari sudut pandangnya, sistem baru saja berbohong dua kali.

Aturan praktis: sebuah task layak dipindahkan ke background bila (1) lambat atau bergantung pada sistem luar, (2) hasilnya tidak dibutuhkan untuk menyusun response, dan (3) boleh terlambat beberapa detik. Email, notifikasi, pembuatan PDF, pengolahan gambar, dan sinkronisasi ke sistem lain hampir selalu memenuhi ketiganya. Memotong kuota kelas tidak memenuhi satu pun.

A.2 Job Queue: Producer, Broker, Worker

Background job membutuhkan tempat menunggu yang aman: bila worker sedang sibuk atau mati, task tidak boleh hilang. Tempat menunggu itu disebut broker. Pada modul ini broker-nya adalah Redis, dan library yang mengatur queue di atas Redis adalah Asynq.

Peran	Di kedai kopi	Di aplikasi kita
Producer	Kasir yang menulis pesanan di kertas	Relay outbox — melakukan enqueue task (asynq.Client)
Broker	Rel penjepit kertas pesanan di depan barista	Redis — menyimpan task sampai diambil
Worker	Barista	Proses worker — mengambil dan mengerjakan task (asynq.Server)

Kabar baiknya, cara kerja Asynq sangat mirip dengan Fiber yang sudah Anda pakai sejak semester lalu. Bedanya hanya pada sumber "permintaan"-nya: Fiber menerima permintaan dari jaringan, Asynq dari Redis.

Fiber (API)	Asynq (worker)	Catatan
fiber.New()	asynq.NewServer(...)	Menyiapkan "server"
app.Post("/enroll", h)	mux.HandleFunc("notification:send", h)	Memetakan nama ke handler
func(c *fiber.Ctx) error	func(ctx context.Context, t *asynq.Task) error	Bentuk handler
Body request (JSON)	t.Payload() — []byte, biasanya JSON	Data yang dibawa
return error → response gagal	return error → task di-retry	Arti mengembalikan error
app.ShutdownWithTimeout	srv.Shutdown()	Berhenti dengan rapi

Setiap task di Asynq memiliki status. Memahami perpindahan status ini sangat membantu ketika berburu kesalahan:

Status	Artinya	Pindah ke
pending	Menunggu diambil worker	active
active	Sedang dikerjakan	completed (berhasil), retry (gagal), archived (gagal, tidak di-retry)
retry	Gagal, menunggu delay sebelum di-retry	active setelah delay-nya habis
archived	Menyerah: jatah attempt habis, atau dinyatakan tidak layak di-retry	— (diperiksa manusia)
completed	Berhasil; disimpan selama masa Retention	dihapus setelah masa Retention habis

Dua nama yang mudah tertukar: JENIS task (task type, misalnya notification:send) menentukan handler mana yang mengerjakannya — seperti URL menentukan handler di Fiber. Nama queue (misalnya notifications) menentukan di "rel" mana task itu menunggu. Worker hanya mengambil task dari queue yang ia dengarkan; task di queue lain akan menunggu selamanya tanpa pesan kesalahan apa pun.

A.3 Masalah Dual Write

Setelah KRS tersimpan, sebuah task harus di-enqueue ke Redis. Masalahnya: PostgreSQL dan Redis adalah dua sistem yang berbeda, dan tidak ada satu transaction pun yang dapat mencakup keduanya. Setiap urutan yang kita pilih meninggalkan celah. Dua percobaan berikut juga diukur pada aplikasi kita:

Urutan	Yang terjadi bila ...	Hasil pengukuran
Commit KRS, lalu masukkan ke Redis	Redis mati sesaat setelah commit	KRS tersimpan, mahasiswa menerima 500, notifikasi tidak pernah dibuat. Bila error-nya diabaikan, notifikasi hilang tanpa jejak.

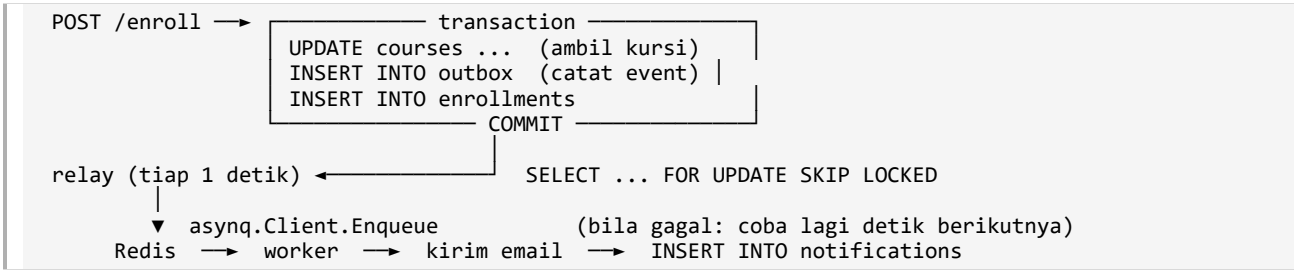
Urutan	Yang terjadi bila ...	Hasil pengukuran
Enqueue ke Redis, di dalam transaction, sebelum commit	Transaction dibatalkan (mis. sudah terdaftar)	Response 409, tetapi task sudah terlanjur di Redis: mahasiswa menerima email "berhasil terdaftar" untuk pendaftaran yang gagal. Pada percobaan kami: 1 pendaftaran, 2 task.

Celah kedua punya varian yang lebih licik: worker bisa mengambil task itu SEBELUM transaction commit, membaca database, dan tidak menemukan apa pun — karena data yang dicarinya belum terlihat oleh transaction lain (ingat isolation level pada Modul 9).

Solusinya: transactional outbox

Idenya sederhana: jangan menulis ke dua sistem. Tulis catatan event ke tabel outbox di PostgreSQL, di dalam transaction yang sama dengan perubahan KRS. Karena satu transaction, keduanya pasti tersimpan bersama atau batal bersama — itulah atomicity dari Modul 9. Sebuah komponen terpisah, relay, kemudian memindahkan isi outbox ke Redis.

Analoginya adalah kotak surat keluar di kantor. Pegawai menaruh surat di kotak itu sebagai bagian dari pekerjaannya; kurir datang berkala mengambil isinya. Bila kantor pos sedang tutup, surat tetap aman di kotak sampai kurir berhasil mengantarnya. Tidak ada surat yang hilang karena kantor pos tutup, dan tidak ada surat yang terkirim untuk pekerjaan yang dibatalkan.



Konsekuensi yang penting: API tidak lagi berbicara dengan Redis sama sekali. Pada percobaan kami, Redis dimatikan, empat mahasiswa mendaftar dan semuanya menerima 201 dalam ± 3 milidetik. Begitu Redis dinyalakan kembali, keempat notifikasi terkirim dalam beberapa detik. Notifikasinya tertunda, tidak hilang.

Relay dan FOR UPDATE SKIP LOCKED

Relay mengambil baris outbox yang belum di-publish dengan SELECT ... FOR UPDATE SKIP LOCKED. FOR UPDATE sudah Anda kenal dari Modul 9: baris yang diambil di-lock. SKIP LOCKED menambahkan satu perilaku: baris yang sedang di-lock pihak lain DILEWATI, tidak ditunggu. Bila dua proses worker berjalan bersamaan, relay keduanya membagi task tanpa saling menunggu dan tanpa mem-publish event yang sama dua kali.

Satu celah masih tersisa: relay bisa mati SETELAH melakukan enqueue task ke Redis tetapi SEBELUM sempat menandai baris outbox. Pada putaran berikutnya, event yang sama akan di-enqueue lagi. Asynq menyediakan penangkalnya: opsi TaskID. Task dengan TaskID yang sudah ada ditolak dengan error asynq.ErrTaskIDConflict, dan bagi relay penolakan itu berarti "sudah pernah di-publish". Perlindungan ini hanya berlaku selama task lama masih tercatat di Redis — karena itu kita menyimpan task yang sudah selesai selama 24 jam dengan opsi Retention.

A.4 At-Least-Once dan Idempotency

Asynq — dan hampir semua sistem queue — menjamin setiap task dijalankan PALING SEDIKIT satu kali (at-least-once), bukan TEPAT satu kali (exactly-once). Task yang sama bisa datang dua kali. Contoh paling sederhana: worker selesai mengirim email, lalu mati sebelum sempat melapor "selesai" ke Redis. Redis tidak tahu emailnya sudah terkirim; setelah lease task itu habis, ia memberikannya ke worker lain. Pada percobaan kami, worker dimatikan paksa di tengah pengiriman, dan task-nya dikerjakan ulang ± 1 menit kemudian oleh worker yang baru dinyalakan.

Karena pengiriman ganda tidak dapat dicegah, yang dapat kita lakukan adalah membuat handler idempoten: menjalankannya dua kali berakibat sama dengan menjalankannya sekali. Caranya dengan idempotency key — sebuah nilai yang unik untuk setiap event. Kita sudah memilikinya: id baris outbox.

Worker mencatat claim ke tabel notifications yang memiliki UNIQUE (outbox_id), lalu mengirim email — keduanya di dalam SATU transaction:

Skenario	Yang terjadi	Hasil
Pengiriman berhasil	Claim di-commit	Satu email, satu baris notifikasi
Pengiriman gagal	Transaction rollback, claim ikut batal	Attempt berikutnya masih dapat mengirim
Task datang lagi setelah berhasil	INSERT ... ON CONFLICT DO NOTHING tidak menyisipkan apa pun	Dilewati, tidak ada email kedua
Dua salinan datang BERSAMAAN	INSERT salinan kedua MENUNGGU sampai salinan pertama commit atau rollback	Satu email. Pada uji 5 salinan serentak: 1 email, 1 baris

Batas yang perlu jujur diakui: bila email sudah terkirim tetapi commit gagal (misalnya database putus tepat di detik itu), claim ikut batal dan attempt berikutnya mengirim email kedua. Celah sekecil ini tidak dapat ditutup dari sisi kita, karena server email adalah sistem lain lagi — persis masalah dual write di A.3. Yang dapat kita lakukan adalah memastikan bahwa kegagalan yang tersisa adalah "email ganda yang jarang" dan bukan "email yang hilang".

Ringkasnya, ada tiga layer perlindungan, masing-masing menutup celah yang berbeda:

Layer	Menutup celah	Diterapkan di
Outbox di dalam transaction KRS	Notifikasi untuk KRS yang batal; notifikasi hilang saat Redis mati	CourseService
TaskID dari id outbox	Relay melakukan enqueue event yang sama dua kali	OutboxRelay
UNIQUE (outbox_id) + transaction claim	Task yang sama dikerjakan dua kali	NotificationWorker

Mengapa worker memakai WithTxOnce, bukan WithTx? WithTx dari Modul 9 melakukan retry seluruh fungsinya bila terjadi deadlock — dan komentarnya sudah memperingatkan: jangan mengirim email di dalamnya, karena email tidak ikut dibatalkan. Di worker, kita memang SENGAJA mengirim email sambil memegang claim, maka kita memakai versi yang tidak melakukan retry dan menyerahkan urusan melakukan

retry kepada Asynq. Konsekuensinya: selama email dikirim, satu koneksi database dipegang. Itu sebabnya jumlah worker serentak (WORKER_CONCURRENCY) harus lebih kecil dari DB_MAX_CONNS.

A.5 Retry, Backoff, dan Permanent Failure

Tidak semua kegagalan layak di-retry:

Golongan	Contoh	Tindakan yang tepat
Temporary failure	Server email sibuk, jaringan putus, database sedang restart	Kembalikan error biasa → Asynq melakukan retry setelah delay
Permanent failure	Alamat email tidak ada, akun sudah dihapus, payload rusak	Bungkus dengan asynq.SkipRetry → langsung masuk archived

- **MaxRetry(5) berarti 6 kali attempt:** satu attempt pertama ditambah lima kali ulang. Kode Asynq memindahkan task ke archived ketika jumlah ulangan sudah mencapai batas, atau ketika error-nya membungkus SkipRetry.
- **Backoff:** delay yang makin panjang di antara attempt, memberi waktu bagi sistem yang sedang bermasalah untuk pulih. Bawaan Asynq memakai rumus $n^4 + 15 + \text{acak}(0..29)\cdot(n+1)$ detik, dengan n = jumlah retry yang sudah terjadi (dimulai dari 0). Retry pertama menunggu 15 sampai 44 detik; retry kelima 271 sampai 416 detik. Untuk praktikum kita memakai 2, 4, 6, 8, dan 10 detik agar hasilnya terlihat dalam hitungan detik. Dalam praktiknya delay itu bisa bertambah sampai 5 detik, karena Asynq memindahkan task dari status retry kembali ke pending secara berkala (DelayedTaskCheckInterval, bawaannya 5 detik).
- **SkipRetry harus DIBUNGKUS, bukan sekadar disebut.** Asynq memeriksanya dengan errors.Is, sama seperti pemeriksaan error pada Modul 7. errors.Is hanya dapat menemukan error yang dibungkus dengan verb %w. Error yang hanya mengandung TEKS "skip retry for the task" tidak dikenali sebagai SkipRetry.
- **Archived bukan tempat sampah.** Ia adalah "meja barang bermasalah" yang harus diperiksa manusia. Task di sana tetap dapat dijalankan ulang secara manual setelah penyebabnya diperbaiki. Jumlahnya terlihat pada endpoint GET /admin/queue yang kita buat.

A.6 Satu Kode, Dua Proses

API dan worker memiliki pola beban yang berbeda. Saat KRS dibuka, API butuh banyak tenaga untuk menerima permintaan; worker cukup sedikit karena email boleh terlambat. Saat malam hari, keduanya hampir menganggur. Karena itu keduanya dijalankan sebagai proses terpisah dari kode sumber yang SAMA, dipilih dengan flag:

```
go run . -role=api      # hanya HTTP
go run . -role=worker   # hanya relay dan worker
go run . -role=all      # keduanya dalam satu proses (bawaan)
```

Ketika worker dimatikan dengan Ctrl+C, Asynq berhenti mengambil task baru dan memberi waktu (ShutdownTimeout) bagi task yang sedang berjalan untuk selesai. Task yang belum selesai dikembalikan ke queue dan akan dikerjakan ulang — satu alasan lagi mengapa handler harus idempoten.

Urutan tidak dijamin. Worker mengerjakan beberapa task sekaligus (WORKER_CONCURRENCY), dan task yang gagal disusul task lain sementara ia menunggu delay retry. Pada uji kami, seorang mahasiswa yang mendaftar, keluar, mendaftar lagi, lalu pindah kelas dalam satu detik menerima email "keluar dari" lebih dulu daripada email "berhasil terdaftar". Karena itu endpoint GET /notifications/mine mengurutkan berdasarkan outbox_id — urutan event — bukan urutan pencatatan notifikasi.

> **Redis juga dapat kehilangan data**
> Outbox menjaga event sampai berhasil diserahkan ke Redis. Setelah itu, keselamatan task bergantung pada Redis. Paket redis-server pada Ubuntu 24.04 yang kami uji memakai pengaturan bawaan appendonly no dengan snapshot berkala (save 3600 1 300 100 60 10000). Bila Redis mati mendadak, task yang masuk setelah snapshot terakhir dapat hilang.
> Untuk praktikum hal ini dapat diterima. Untuk production, aktifkan appendonly yes dan pelajari bagian Persistence pada dokumentasi Redis sebelum memutuskan.

B. Langkah Panduan

Lanjutkan proyek `latihan-fiber` dari Modul 10. PostgreSQL dan MongoDB tetap dipakai seperti sebelumnya; pertemuan ini menambahkan Redis.

B.0 Cara Berburu Kesalahan pada Sistem Asinkron

Urutan	Yang dikerjakan	Mengapa
1. Dengarkan compiler	Jalankan go build dan go vet, baca pesannya sampai selesai	Tiga kesalahan tumbang di sini, satu per satu, karena Go berhenti di package pertama yang gagal. Baris yang ditunjuk compiler tidak selalu baris yang salah
2. Bandingkan komentar dengan kode	Komentar di modul ini dijamin benar	Kode yang bertentangan dengan komentarnya adalah tersangka utama
3. Ikuti jejak satu event	outbox → Redis → worker → notifications → logs/mail.log	Pada sistem asinkron, gejala muncul jauh dari penyebabnya. Cari di titik mana jejaknya terputus
4. Gagalkan sesuatu dengan sengaja	Matikan Redis, aktifkan MAILER_FAIL_EVERY, kirim task ganda	Separuh kesalahan hanya terlihat ketika ada yang gagal
5. Hitung, jangan menebak	Bandingkan jumlah baris outbox, notifications, dan baris logs/mail.log	Tiga angka yang seharusnya sama adalah alat ukur paling jujur

Query monitoring berikut menampilkan jejak setiap event. Simpan, karena Anda akan sering menjalankannya:

```
SELECT o.id, o.event_type, o.published_at IS NOT NULL AS di-publish,
       n.subject, n.sent_at
FROM outbox o
LEFT JOIN notifications n ON n.outbox_id = o.id
ORDER BY o.id DESC
LIMIT 10;
```

Satu petunjuk umum: angka yang tidak berubah sama mencurigakannya dengan angka yang salah. Bila outbox sudah di-publish tetapi notifikasi tidak kunjung muncul, task-nya sedang menunggu di suatu tempat. Endpoint GET /admin/queue (Langkah 10) memberi tahu di mana.

LANGKAH 0 — Memasang Redis (dikerjakan sebelum praktikum)

Asynq membutuhkan Redis versi 4.0 atau lebih baru. Pilih satu cara sesuai sistem operasi Anda. Apa pun caranya, hasil akhirnya sama: sebuah server Redis yang mendengarkan di localhost port 6379.

0.1 Windows — Memurai Developer

Dokumentasi instalasi resmi Redis hanya menyediakan jalur Docker untuk Windows. Agar tidak perlu memasang Docker, kita memakai Memurai: server yang kompatibel dengan Redis dan berjalan langsung sebagai service Windows. Redis sendiri menerbitkan tutorial pemakaian Memurai di situsnya. Edisi Developer gratis untuk pengembangan. Bila Anda sudah terbiasa dengan Docker atau WSL, keduanya juga dapat dipakai — yang penting Redis dapat dihubungi di localhost:6379.

1. Unduh Memurai Developer dari https://www.memurai.com (menu download, pilih edisi Developer). Berkasnya berupa installer .msi. Bila Anda memakai winget, dapat juga: winget install -e --id Memurai.MemuraiDeveloper
2. Jalankan installer dengan hak administrator. Biarkan pilihan bawaan: dipasang sebagai Windows service, port 6379, folder instalasi ditambahkan ke PATH.
3. Service Memurai langsung berjalan dan otomatis menyala setiap Windows dinyalakan.

Periksa di Command Prompt yang BARU dibuka (agar PATH terbaru terbaca):

```
memurai-cli ping
PONG
```

Untuk mematikan dan menyalakan service (dibutuhkan pada Bagian C):

```
memurai.exe --service-stop
memurai.exe --service-start

# alternatif, dari Command Prompt yang dijalankan sebagai administrator
net stop memurai
net start memurai
```

Batasan edisi Developer

Menurut situs Memurai, edisi Developer hanya untuk pengembangan dan pengujian (tidak boleh untuk production), dan perlu dijalankan ulang setelah beberapa hari menyala terus-menerus. Bila suatu saat aplikasi tiba-tiba tidak dapat terhubung ke Redis, jalankan perintah memurai.exe dengan opsi service-start (lihat di atas) lebih dulu.

Syarat sistem menurut dokumentasi Memurai: Windows 10 atau Windows Server 2012 ke atas, 64-bit.

0.2 macOS — Homebrew

```
brew install redis
brew services start redis
brew services list        # redis harus berstatus started

redis-cli ping
PONG
```

Untuk mematikan dan menyalakan: brew services stop redis dan brew services start redis.

0.3 Linux — Ubuntu 24.04 / 22.04

Cara paling sederhana adalah paket bawaan Ubuntu:

```
sudo apt update
sudo apt install redis-server

sudo systemctl status redis-server     # harus active (running)
redis-cli ping
PONG
```

Paket bawaan Ubuntu tidak selalu versi terbaru (pada Ubuntu 24.04 yang kami uji: Redis 7.0.15), tetapi jauh di atas syarat minimum Asynq. Bila Anda menginginkan versi terbaru, dokumentasi Redis menyediakan repository resminya:

```
sudo apt-get install lsb-release curl gpg
curl -fsSL https://packages.redis.io/gpg | sudo gpg --dearmor -o /usr/share/keyrings/redis-archive-
keyring.gpg
sudo chmod 644 /usr/share/keyrings/redis-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/redis-archive-keyring.gpg] https://packages.redis.io/deb
$(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/redis.list
sudo apt-get update
sudo apt-get install redis
```

Untuk mematikan dan menyalakan: sudo systemctl stop redis-server dan sudo systemctl start redis-server.

> **Keamanan: Redis bukan untuk dibuka ke jaringan**
>
> Redis yang kita pasang tidak memakai password dan hanya listening di localhost. Itu aman selama tetap di localhost. Redis tanpa password yang dapat diakses dari jaringan dapat dibaca, diubah, bahkan dikosongkan oleh siapa pun.
>
> Jangan mengubah pengaturan bind atau protected-mode di file konfigurasi Redis untuk praktikum ini. Di server sungguhan, Redis diberi password (REDIS_PASSWORD pada .env kita sudah disiapkan untuk itu) dan ditempatkan di jaringan internal.

LANGKAH 1 — Dependency dan Konfigurasi

```
go get github.com/hibiken/asynq@v0.26.0
```

Asynq ikut memasang go-redis sebagai dependency-nya. Versi ditulis eksplisit karena Asynq masih berada pada versi 0.x: menurut README-nya, API publiknya dapat berubah sebelum versi 1.0. Tambahkan baris berikut ke .env:

▸ **.env (tambahan)**

```
REDIS_ADDR=localhost:6379
REDIS_PASSWORD=
REDIS_DB=0

WORKER_CONCURRENCY=5
RELAY_INTERVAL_MS=1000
RELAY_BATCH=100
MAILER_DELAY_MS=1500
MAILER_FAIL_EVERY=0
```

Variabel	Kegunaan
REDIS_ADDR, REDIS_PASSWORD, REDIS_DB	Alamat Redis. REDIS_PASSWORD dikosongkan untuk Redis lokal
WORKER_CONCURRENCY	Jumlah task yang dikerjakan bersamaan oleh satu proses worker
RELAY_INTERVAL_MS, RELAY_BATCH	Seberapa sering relay memeriksa outbox, dan berapa baris sekali ambil
MAILER_DELAY_MS	Lama satu pengiriman email tiruan
MAILER_FAIL_EVERY	Bila diisi N, setiap attempt pengiriman ke-N gagal sementara. 0 = tidak pernah gagal

LANGKAH 2 — Skema Outbox dan Notifikasi

▸ **migrations/008_outbox.sql**

```
-- Migration 008: transactional outbox dan catatan notifikasi
--   psql -U postgres -d praktikum_backend -f migrations/008_outbox.sql

-- ---------------------------------------------------------------------
-- outbox — "kotak surat keluar" milik database
--
-- Setiap event penting (mendaftar, keluar, pindah kelas) dicatat di
-- sini DI DALAM transaction yang sama dengan perubahan datanya. Karena
-- satu transaction, hanya ada dua kemungkinan: perubahan data dan
-- catatan event-nya sama-sama tersimpan, atau sama-sama batal.
-- ---------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS outbox (
    id           BIGSERIAL   PRIMARY KEY,
    event_type   VARCHAR(50) NOT NULL,
    payload      JSONB       NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    -- NULL berarti belum di-publish ke queue.
    published_at TIMESTAMPTZ NULL
);

-- Relay hanya pernah mencari baris yang belum di-publish. Partial index
-- membuat index ini tetap kecil walaupun tabel outbox tumbuh jutaan baris,
-- karena baris yang sudah di-publish tidak ikut diindeks.
CREATE INDEX IF NOT EXISTS outbox_unpublished_idx
    ON outbox (id) WHERE published_at IS NULL;

-- ---------------------------------------------------------------------
-- notifications — bukti bahwa sebuah event SUDAH dikirimkan
--
-- UNIQUE (outbox_id) adalah idempotency key: satu event hanya boleh
-- menghasilkan satu notifikasi, berapa kali pun task-nya dijalankan.
--
-- outbox_id sengaja TIDAK diberi FOREIGN KEY ke outbox. Baris outbox yang
-- sudah lama di-publish suatu saat perlu dibersihkan, sedangkan riwayat
-- notifikasi milik mahasiswa harus tetap ada.
-- ---------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS notifications (
    id         BIGSERIAL    PRIMARY KEY,
    outbox_id  BIGINT       NOT NULL,
    user_id    INTEGER      NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    channel    VARCHAR(20)  NOT NULL,
    subject    VARCHAR(200) NOT NULL,
    body       TEXT         NOT NULL,
    sent_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),

    CONSTRAINT notifications_outbox_unique UNIQUE (outbox_id)
);
```

```
CREATE INDEX IF NOT EXISTS notifications_user_idx ON notifications (user_id, outbox_id DESC);

INSERT INTO permissions (name, description) VALUES
    ('queue:monitor', 'Melihat keadaan outbox dan queue task')
ON CONFLICT (name) DO NOTHING;

INSERT INTO role_permissions (role_name, permission_name) VALUES
    ('admin', 'queue:monitor')
ON CONFLICT DO NOTHING;
```

```
psql -U postgres -d praktikum_backend -f migrations/008_outbox.sql
```

Tiga keputusan perancangan yang perlu dipahami:

- **Partial index.** Relay hanya mencari baris yang published_at-nya NULL. Index biasa akan ikut menyimpan jutaan baris yang sudah lama di-publish; partial index hanya menyimpan yang belum.
- **UNIQUE (outbox_id) sebagai idempotency key.** Inilah yang membuat worker aman dijalankan berulang kali (A.4). Tanpa constraint ini, idempotency hanya bergantung pada kehati-hatian kode — dan Modul 9 sudah menunjukkan bahwa pemeriksaan di kode saja mudah kalah oleh konkurensi.
- **outbox_id tanpa FOREIGN KEY.** Ini keputusan sengaja yang dijelaskan di komentar file. Outbox adalah tabel "lalu lintas" yang suatu saat dibersihkan, sedangkan riwayat notifikasi milik mahasiswa.

LANGKAH 3 — Model

▸ **app/model/notification.go**

```
package model

import (
    "encoding/json"
    "time"
)

// Nama-nama event yang dicatat ke outbox. Ditulis sebagai konstanta,
// bukan string yang diketik ulang di banyak tempat, karena salah ketik pada
// nama event tidak ditangkap compiler maupun database.
const (
    EventEnrollmentCreated = "enrollment.created"
    EventEnrollmentDropped = "enrollment.dropped"
    EventEnrollmentSwapped = "enrollment.swapped"
)

// TaskSendNotification adalah nama JENIS task (task type); worker memakainya untuk
// memilih handler. QueueNotifications adalah nama QUEUE tempat task
// itu menunggu; worker hanya mengambil task dari queue yang ia dengarkan.
const (
    TaskSendNotification = "notification:send"
    QueueNotifications   = "notifications"
)

// EnrollmentEvent adalah isi sebuah event KRS.
//
// Nama dan kode kelas DISALIN ke dalam event, bukan hanya id-nya.
// Event adalah catatan tentang apa yang terjadi PADA SAAT ITU. Bila nama
// kelas diubah esok hari, notifikasi untuk event hari ini tetap memakai
// nama yang dilihat mahasiswa ketika ia mendaftar.
type EnrollmentEvent struct {
    UserID         int    `json:"user_id"`
    CourseID       int    `json:"course_id"`
    CourseCode     string `json:"course_code"`
    CourseName     string `json:"course_name"`
    FromCourseCode string `json:"from_course_code,omitempty"`
    FromCourseName string `json:"from_course_name,omitempty"`
}
```

```
// OutboxEvent adalah satu baris tabel outbox.
type OutboxEvent struct {
    ID        int64
    EventType string
    Payload   []byte
    CreatedAt time.Time
}

// NotificationJob adalah isi task yang dikirim ke queue.
//
// OutboxID ikut dibawa karena dialah idempotency key: worker memakainya
// untuk mengetahui apakah event ini sudah pernah dikirimkan.
//
// Payload dibiarkan sebagai JSON mentah (json.RawMessage). Relay cukup
// meneruskannya apa adanya tanpa perlu tahu bentuk setiap jenis event;
// yang membaca isinya adalah worker, berdasarkan EventType.
type NotificationJob struct {
    OutboxID  int64           `json:"outbox_id"`
    EventType string          `json:"event_type"`
    Payload   json.RawMessage `json:"payload"`
}

// Message adalah pesan yang siap dikirim, tidak bergantung pada cara
// pengirimannya (email, push notification, dan seterusnya).
type Message struct {
    To      string
    Subject string
    Body    string
}

// Notification adalah notifikasi yang sudah terkirim, seperti yang dilihat
// mahasiswa pada GET /notifications/mine.
type Notification struct {
    ID       int64     `json:"id"`
    OutboxID int64     `json:"outbox_id"`
    UserID   int       `json:"user_id"`
    Channel  string    `json:"channel"`
    Subject  string    `json:"subject"`
    Body     string    `json:"body"`
    SentAt   time.Time `json:"sent_at"`
}

// QueueStats adalah ringkasan keadaan untuk admin.
type QueueStats struct {
    OutboxPending     int        `json:"outbox_pending"`
    OutboxPublished   int        `json:"outbox_published"`
    OldestPendingAge  string     `json:"oldest_pending_age"`
    NotificationsSent int        `json:"notifications_sent"`
    Queue             *QueueInfo `json:"queue"`
    QueueError        string     `json:"queue_error,omitempty"`
}

// QueueInfo adalah salinan sebagian informasi queue dari Asynq. Ditulis
// ulang di sini agar package model tetap tidak bergantung pada library apa pun.
type QueueInfo struct {
    Name      string `json:"name"`
    Pending   int    `json:"pending"`
    Active    int    `json:"active"`
    Scheduled int    `json:"scheduled"`
    Retry     int    `json:"retry"`
    Archived  int    `json:"archived"`
    Completed int    `json:"completed"`
    Processed int    `json:"processed_today"`
    Failed    int    `json:"failed_today"`
}
```

Package model tetap tidak mengimpor library luar mana pun. QueueInfo sengaja ditulis ulang di sini, bukan memakai asynq.QueueInfo secara langsung, agar response API tidak berubah bentuk bila suatu hari Asynq mengubah struct-nya.

LANGKAH 4 — Mailer Tiruan

Praktikum ini tidak mengirim email sungguhan. LogMailer menulis pesan ke logs/mail.log, tetapi meniru dua sifat server email sungguhan yang justru menjadi inti pertemuan ini: lambat, dan sesekali gagal.

▸ **helper/mailer.go**

```
package helper

import (
    "context"
    "errors"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "sync"
    "sync/atomic"
    "time"

    "latihan-fiber/app/model"
)

// Dua golongan kegagalan pengiriman. Pembedaan ini menentukan apakah
// sebuah task layak di-retry.
var (
    // ErrTemporaryDelivery: server email sedang sibuk atau jaringan putus.
    // Melakukan retry beberapa saat kemudian kemungkinan besar berhasil.
    ErrTemporaryDelivery = errors.New("pengiriman gagal sementara")

    // ErrPermanentDelivery: alamat tujuan tidak ada. Melakukan retry seribu
    // kali pun hasilnya akan tetap sama.
    ErrPermanentDelivery = errors.New("pengiriman gagal permanen")
)

// Mailer adalah kemampuan mengirim pesan. Service hanya mengenal interface
// ini, sehingga LogMailer dapat diganti mailer SMTP sungguhan tanpa
// mengubah satu baris pun kode service.
type Mailer interface {
    Send(ctx context.Context, msg model.Message) error
}

// LogMailer adalah mailer tiruan untuk praktikum. Ia tidak mengirim email
// sungguhan, tetapi menulis pesannya ke logs/mail.log dan meniru dua sifat
// server email sungguhan: lambat, dan sesekali gagal.
type LogMailer struct {
    mu        sync.Mutex
    path      string
    delay     time.Duration
    failEvery int64
    attempts  atomic.Int64
}

// NewLogMailer membuat LogMailer.
//
//   - delay     : lama satu kali pengiriman, meniru server email yang lambat.
//   - failEvery : setiap attempt ke-N gagal sementara. Nol berarti tidak
//     pernah gagal.
func NewLogMailer(path string, delay time.Duration, failEvery int) (Mailer, error) {
    if err := os.MkdirAll(filepath.Dir(path), 0o755); err != nil {
        return nil, fmt.Errorf("menyiapkan folder log email: %w", err)
    }
    // Pointer, bukan nilai: mutex dan counter attempt harus dimiliki
    // bersama oleh seluruh goroutine worker, bukan disalin per pemanggilan.
    return LogMailer{path: path, delay: delay, failEvery: int64(failEvery)}, nil
}

func (m *LogMailer) Send(ctx context.Context, msg model.Message) error {
    n := m.attempts.Add(1)
```

```
    // Meniru waktu tempuh ke server email. Bila context dibatalkan lebih
    // dulu (timeout task habis, atau worker sedang dimatikan),
    // pengiriman dihentikan saat itu juga.
    select {
    case <-ctx.Done():
        return ctx.Err()
    case <-time.After(m.delay):
    }

    // Domain .invalid dicadangkan oleh RFC 2606 dan dijamin tidak pernah
    // ada. Kita memakainya untuk meniru "alamat tujuan tidak ditemukan".
    if strings.HasSuffix(strings.ToLower(msg.To), ".invalid") {
        return fmt.Errorf("%w: alamat %s tidak ditemukan", ErrPermanentDelivery, msg.To)
    }
    if m.failEvery > 0 && n%m.failEvery == 0 {
        return fmt.Errorf("%w: server email sibuk (attempt ke-%d)", ErrTemporaryDelivery, n)
    }

    m.mu.Lock()
    defer m.mu.Unlock()

    f, err := os.OpenFile(m.path, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0o600)
    if err != nil {
        return fmt.Errorf("%w: %v", ErrTemporaryDelivery, err)
    }
    defer f.Close()

    // Hanya alamat dan subjek yang dicatat. Isi pesan dapat memuat data
    // pribadi yang tidak perlu ikut tersimpan di file log.
    _, err = fmt.Fprintf(f, "%s  to=%s  subject=%q\n",
        time.Now().Format(time.RFC3339), msg.To, msg.Subject)
    return err
}
```

Perhatikan bahwa service nantinya hanya mengenal interface `Mailer`. Mengganti LogMailer dengan mailer SMTP sungguhan cukup dilakukan di satu tempat: fungsi pembuatnya. Prinsip yang sama dengan repository sejak Modul 4.

LANGKAH 5 — Repository Outbox dan Notifikasi

▸ **app/repository/outbox_repository.go**

```
package repository

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "latihan-fiber/app/model"
)

type OutboxRepository interface {
    WithTx(tx DBTX) OutboxRepository

    // Insert WAJIB dipanggil lewat repository hasil WithTx, di dalam
    // transaction yang sama dengan perubahan data yang dicatatnya.
    Insert(ctx context.Context, eventType string, event any) error

    // FetchUnpublished mengunci baris yang diambilnya sampai transaction
    // selesai. Karena itu ia hanya bermakna bila dipanggil di dalam
    // transaction.
    FetchUnpublished(ctx context.Context, limit int) ([]model.OutboxEvent, error)
    MarkPublished(ctx context.Context, ids []int64) error
```

```
    Stats(ctx context.Context) (pending, published int, oldest time.Duration, err error)
}

type outboxPostgresRepository struct{ db DBTX }

func NewOutboxRepository(db DBTX) OutboxRepository {
    return &outboxPostgresRepository{db: db}
}

func (r *outboxPostgresRepository) WithTx(tx DBTX) OutboxRepository {
    return &outboxPostgresRepository{db: tx}
}

func (r *outboxPostgresRepository) Insert(ctx context.Context, eventType string, event any) error {
    payload, err := json.Marshal(event)
    if err != nil {
        return fmt.Errorf("menyusun payload outbox: %w", err)
    }
    _, err = r.db.Exec(ctx,
        `INSERT INTO outbox (event_type, payload) VALUES ($1, $2)`,
        eventType, payload)
    if err != nil {
        return fmt.Errorf("mencatat outbox: %w", err)
    }
    return nil
}

// FetchUnpublished memakai FOR UPDATE SKIP LOCKED.
//
//   - FOR UPDATE  : baris yang diambil di-lock, sehingga relay lain tidak
//     dapat mengambil baris yang sama pada saat yang sama.
//   - SKIP LOCKED : baris yang sedang di-lock relay lain DILEWATI, bukan
//     ditunggu. Dua relay yang berjalan bersamaan masing-masing mendapat
//     bagian yang berbeda, tidak saling menunggu.
func (r *outboxPostgresRepository) FetchUnpublished(ctx context.Context, limit int)
([]model.OutboxEvent, error) {
    rows, err := r.db.Query(ctx, `
        SELECT id, event_type, payload, created_at
        FROM outbox
        WHERE published_at IS NULL
        ORDER BY id
        LIMIT $1
        FOR UPDATE SKIP LOCKED`, limit)
    if err != nil {
        return nil, fmt.Errorf("mengambil outbox: %w", err)
    }
    defer rows.Close()

    events := []model.OutboxEvent{}
    for rows.Next() {
        var e model.OutboxEvent
        if err := rows.Scan(&e.ID, &e.EventType, &e.Payload, &e.CreatedAt); err != nil {
            return nil, fmt.Errorf("membaca row outbox: %w", err)
        }
        events = append(events, e)
    }
    return events, rows.Err()
}

func (r *outboxPostgresRepository) MarkPublished(ctx context.Context, ids []int64) error {
    if len(ids) == 0 {
        return nil
    }
    _, err := r.db.Exec(ctx,
        `UPDATE outbox SET published_at = NOW() WHERE id = ANY($1)`, ids)
    if err != nil {
        return fmt.Errorf("menandai outbox: %w", err)
    }
    return nil
}

func (r *outboxPostgresRepository) Stats(ctx context.Context) (int, int, time.Duration, error) {
```

```
    var pending, published int
    var oldestSeconds float64
    err := r.db.QueryRow(ctx, `
        SELECT
            COUNT(*) FILTER (WHERE published_at IS NULL),
            COUNT(*) FILTER (WHERE published_at IS NOT NULL),
            COALESCE(EXTRACT(EPOCH FROM NOW() - MIN(created_at) FILTER (WHERE published_at IS NULL)),
0)
        FROM outbox`).Scan(&pending, &published, &oldestSeconds)
    if err != nil {
        return 0, 0, 0, fmt.Errorf("menghitung outbox: %w", err)
    }
    return pending, published, time.Duration(oldestSeconds * float64(time.Second)), nil
}
```

▸ **app/repository/notification_repository.go**

```
package repository

import (
    "context"
    "errors"
    "fmt"

    "github.com/jackc/pgx/v5"

    "latihan-fiber/app/model"
)

type NotificationRepository interface {
    WithTx(tx DBTX) NotificationRepository

    // Claim mencatat bahwa notifikasi untuk sebuah event akan dikirim.
    // Mengembalikan false bila event itu sudah pernah tercatat.
    Claim(ctx context.Context, n model.Notification) (bool, error)

    FindByUser(ctx context.Context, userID, limit int) ([]model.Notification, error)
    Count(ctx context.Context) (int, error)
}

type notificationPostgresRepository struct{ db DBTX }

func NewNotificationRepository(db DBTX) NotificationRepository {
    return &notificationPostgresRepository{db: db}
}

func (r *notificationPostgresRepository) WithTx(tx DBTX) NotificationRepository {
    return &notificationPostgresRepository{db: tx}
}

// Claim memakai ON CONFLICT DO NOTHING, bukan "SELECT dulu, lalu INSERT
// bila belum ada". Versi SELECT-lalu-INSERT adalah pola check-then-act dari
// Modul 9: dua worker dapat sama-sama melihat "belum ada" pada detik yang
// sama. UNIQUE constraint memang tetap menolak INSERT yang kedua, tetapi
// penolakan itu datang sebagai error 23505 — task-nya dianggap GAGAL dan
// di-retry, padahal yang benar adalah "sudah dikirim, lewati saja".
//
// Dengan ON CONFLICT, bila worker lain sedang memegang baris dengan
// outbox_id yang sama tetapi belum commit, pernyataan ini MENUNGGU sampai
// worker itu selesai — lalu melewatinya bila worker itu commit, atau
// melanjutkan bila worker itu rollback.
func (r *notificationPostgresRepository) Claim(ctx context.Context, n model.Notification) (bool,
error) {
    var id int64
    err := r.db.QueryRow(ctx, `
        INSERT INTO notifications (outbox_id, user_id, channel, subject, body)
        VALUES ($1, $2, $3, $4, $5)
        ON CONFLICT (outbox_id) DO NOTHING
        RETURNING id`,
        n.OutboxID, n.UserID, n.Channel, n.Subject, n.Body).Scan(&id)
    if errors.Is(err, pgx.ErrNoRows) {
```

```
        return false, nil
    }
    if err != nil {
        return false, fmt.Errorf("mencatat notifikasi: %w", err)
    }
    return true, nil
}

// FindByUser mengurutkan berdasarkan outbox_id, bukan id notifikasi.
// Worker mengerjakan beberapa task sekaligus, sehingga urutan
// notifikasi DICATAT tidak sama dengan urutan event-nya. outbox_id
// mengikuti urutan event, karena seluruh perubahan KRS milik satu
// mahasiswa berjalan bergiliran (LockStudent, Modul 9).
func (r *notificationPostgresRepository) FindByUser(ctx context.Context, userID, limit int)
([]model.Notification, error) {
    rows, err := r.db.Query(ctx, `
        SELECT id, outbox_id, user_id, channel, subject, body, sent_at
        FROM notifications
        WHERE user_id = $1
        ORDER BY outbox_id DESC
        LIMIT $2`, userID, limit)
    if err != nil {
        return nil, fmt.Errorf("mengambil notifikasi: %w", err)
    }
    defer rows.Close()

    result := []model.Notification{}
    for rows.Next() {
        var n model.Notification
        if err := rows.Scan(&n.ID, &n.OutboxID, &n.UserID, &n.Channel,
            &n.Subject, &n.Body, &n.SentAt); err != nil {
            return nil, fmt.Errorf("membaca row notifikasi: %w", err)
        }
        result = append(result, n)
    }
    return result, rows.Err()
}

func (r *notificationPostgresRepository) Count(ctx context.Context) (int, error) {
    var total int
    if err := r.db.QueryRow(ctx, `SELECT COUNT(*) FROM notifications`).Scan(&total); err != nil {
        return 0, fmt.Errorf("menghitung notifikasi: %w", err)
    }
    return total, nil
}
```

Tambahkan method berikut pada TxManager, tepat di atas method runOnce:

▸ **app/repository/tx.go (tambahan)**

```
// WithTxOnce sama dengan WithTx, tetapi TIDAK PERNAH melakukan retry.
//
// Dipakai ketika fn memang harus melakukan sesuatu di luar database —
// mengirim email, melakukan enqueue task — sambil memegang lock
// di database. Melakukan retry fn berarti mengulang tindakan luar itu.
// Tanggung jawab melakukan retry dipindahkan ke pemanggil: worker queue
// sudah punya mekanisme retry sendiri, dan relay outbox akan melakukan retry
// pada putaran berikutnya.
func (m *TxManager) WithTxOnce(ctx context.Context, fn func(tx DBTX) error) error {
    return m.runOnce(ctx, fn)
}
```

LANGKAH 6 — Mencatat Event di Dalam Transaction KRS

CourseService kini membutuhkan OutboxRepository. Ganti struct dan constructor-nya:

▸ **app/service/course_service.go — struct dan constructor**

```
type CourseService struct {
    courses repository.CourseRepository
    outbox  repository.OutboxRepository
```

```
    tx      *repository.TxManager
}

func NewCourseService(
    courses repository.CourseRepository,
    outbox repository.OutboxRepository,
    tx *repository.TxManager,
) *CourseService {
    return &CourseService{courses: courses, outbox: outbox, tx: tx}
}
```

Pada method Enroll, ganti bagian di dalam WithTx mulai dari langkah 3 sampai akhir fungsi transaction-nya:

▸ **app/service/course_service.go — Enroll**

```
        // 3. Ambil satu kursi secara atomik.
        enrolled, err = repo.IncrementEnrolled(ctx, courseID)
        if err != nil {
            return err
        }

        // 4. Catat event-nya ke outbox, memakai transaction yang sama.
        //    Urutan di dalam transaction tidak mengubah hasil akhir: bila
        //    langkah 5 gagal, catatan ini ikut dibatalkan bersama
        //    penambahan kursi pada langkah 3.
        if err := s.outbox.Insert(ctx, model.EventEnrollmentCreated, model.EnrollmentEvent{
            UserID: current.UserID, CourseID: enrolled.ID,
            CourseCode: enrolled.Code, CourseName: enrolled.Name,
        }); err != nil {
            return err
        }

        // 5. Catat pendaftarannya. Bila ternyata sudah terdaftar, error
        //    di sini membatalkan seluruh langkah sebelumnya.
        return repo.InsertEnrollment(ctx, courseID, current.UserID)
    })
```

Pada method Drop, ganti bagian setelah DeleteEnrollment:

▸ **app/service/course_service.go — Drop**

```
        if err := repo.DeleteEnrollment(ctx, courseID, current.UserID); err != nil {
            return err
        }
        course, err := repo.DecrementEnrolled(ctx, courseID)
        if err != nil {
            return err
        }
        return s.outbox.WithTx(tx).Insert(ctx, model.EventEnrollmentCreated, model.EnrollmentEvent{
            UserID: current.UserID, CourseID: course.ID,
            CourseCode: course.Code, CourseName: course.Name,
        })
    })
```

Pada method Swap, ganti bagian mulai dari IncrementEnrolled untuk kelas tujuan:

▸ **app/service/course_service.go — Swap**

```
        joined, err = repo.IncrementEnrolled(ctx, req.ToCourseID)
        if err != nil {
            return err
        }
        if err := repo.InsertEnrollment(ctx, req.ToCourseID, current.UserID); err != nil {
            return err
        }

        // Satu event "pindah", bukan dua event "keluar" dan "masuk".
        // Mahasiswa melakukan satu tindakan, maka ia menerima satu notifikasi.
        return s.outbox.WithTx(tx).Insert(ctx, model.EventEnrollmentSwapped, model.EnrollmentEvent{
            UserID: current.UserID, CourseID: to.ID,
```

```
            CourseCode: to.Code, CourseName: to.Name,
            FromCourseCode: from.Code, FromCourseName: from.Name,
        })
    })
```

Checkpoint pertama

main.go belum diperbarui sehingga belum dapat dikompilasi seluruhnya. Periksa package yang sudah selesai saja:

```
go build ./app/... ./helper/...
go vet ./app/... ./helper/...
```

Lanjutkan ke langkah berikutnya hanya bila kedua perintah di atas bersih.

LANGKAH 7 — Relay Outbox

▸ **app/service/outbox_relay.go**

```
package service

import (
    "context"
    "errors"
    "fmt"
    "log/slog"
    "time"

    "github.com/hibiken/asynq"

    "latihan-fiber/app/model"
    "latihan-fiber/app/repository"
)

// TaskEnqueuer adalah kemampuan melakukan enqueue task.
// *asynq.Client memenuhi interface ini.
type TaskEnqueuer interface {
    EnqueueContext(ctx context.Context, task *asynq.Task, opts ...asynq.Option) (*asynq.TaskInfo,
error)
}

// OutboxRelay memindahkan event dari tabel outbox ke queue Redis.
//
// Relay adalah satu-satunya komponen yang membaca outbox. API tidak pernah
// berbicara dengan Redis; ia cukup menulis ke PostgreSQL. Karena itu
// mahasiswa tetap dapat mengisi KRS walaupun Redis sedang mati — notifikasinya
// hanya tertunda, tidak hilang.
type OutboxRelay struct {
    outbox   repository.OutboxRepository
    tx       *repository.TxManager
    queue    TaskEnqueuer
    logger   *slog.Logger
    interval time.Duration
    batch    int
}

func NewOutboxRelay(
    outbox repository.OutboxRepository,
    tx *repository.TxManager,
    queue TaskEnqueuer,
    logger *slog.Logger,
    interval time.Duration,
    batch int,
) *OutboxRelay {
    return &OutboxRelay{outbox: outbox, tx: tx, queue: queue, logger: logger,
        interval: interval, batch: batch}
}
```

```
// Run berjalan terus sampai ctx dibatalkan.
func (r *OutboxRelay) Run(ctx context.Context) {
    ticker := time.NewTicker(r.interval)
    defer ticker.Stop()

    for {
        n, err := r.RelayOnce(ctx)
        if err != nil {
            r.logger.Warn("relay_delayed",
                slog.Int("published", n), slog.String("error", err.Error()))
        } else if n > 0 {
            r.logger.Info("relay_done", slog.Int("published", n))
        }

        // Satu batch penuh berarti kemungkinan besar masih ada sisa:
        // langsung lanjutkan tanpa menunggu. Selain itu, tunggu giliran
        // berikutnya.
        if err == nil && n == r.batch && ctx.Err() == nil {
            continue
        }
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
        }
    }
}

// RelayOnce mem-publish paling banyak satu batch. Mengembalikan jumlah
// event yang berhasil di-publish.
func (r *OutboxRelay) RelayOnce(ctx context.Context) (int, error) {
    var published int
    var enqueueErr error

    err := r.tx.WithTxOnce(ctx, func(tx repository.DBTX) error {
        repo := r.outbox.WithTx(tx)

        events, err := repo.FetchUnpublished(ctx, r.batch)
        if err != nil {
            return err
        }

        ids := make([]int64, 0, len(events))
        for _, e := range events {
            // Berhenti pada kegagalan pertama. Bila Redis mati, event
            // berikutnya pun pasti gagal; tidak ada gunanya dicoba satu per
            // satu. Yang belum di-publish tetap NULL dan diambil lagi pada
            // putaran berikutnya.
            ids = append(ids, e.ID)
            if err := r.publish(ctx, e); err != nil {
                enqueueErr = err
                break
            }
        }

        // Hanya event yang SUDAH berhasil di-enqueue yang ditandai.
        if err := repo.MarkPublished(ctx, ids); err != nil {
            return err
        }
        published = len(ids)
        return nil
    })
    if err != nil {
        return 0, err
    }
    return published, enqueueErr
}

func (r *OutboxRelay) publish(ctx context.Context, e model.OutboxEvent) error {
    job := model.NotificationJob{
        OutboxID: e.ID, EventType: e.EventType, Payload: e.Payload,
    }
```

```
    task := asynq.NewTask(model.TaskSendNotification, job,
        // TaskID yang diturunkan dari id outbox membuat enqueue ulang
        // event yang sama ditolak Redis. Ini menutup celah ketika relay
        // mati SETELAH melakukan enqueue tetapi SEBELUM sempat commit.
        asynq.TaskID(fmt.Sprintf("outbox-%d", e.ID)),
        asynq.Queue(model.QueueNotifications),
        asynq.MaxRetry(5),
        asynq.Timeout(30*time.Second),
        // Task yang selesai disimpan 24 jam. Selama itu, TaskID-nya
        // tetap tercatat sehingga perlindungan di atas tetap berlaku.
        asynq.Retention(24*time.Hour),
    )

    _, err := r.queue.EnqueueContext(ctx, task)
    if errors.Is(err, asynq.ErrTaskIDConflict) {
        // Sudah pernah di-enqueue pada putaran sebelumnya. Bagi relay,
        // itu sama dengan berhasil.
        return nil
    }
    if err != nil {
        return fmt.Errorf("enqueue outbox %d: %w", e.ID, err)
    }
    return nil
}
```

Relay menahan lock row outbox selama melakukan enqueue task ke Redis. Ini salah satu tempat langka di mana menunggu sistem luar di dalam transaction memang disengaja: lock-nya hanya pada row outbox yang tidak disentuh siapa pun selain relay, bukan pada baris kelas yang diperebutkan mahasiswa. Bandingkan dengan baris pertama tabel di A.1.

LANGKAH 8 — Worker Notifikasi

Penyusunan isi pesan dipisahkan sebagai fungsi murni, seperti aturan bisnis sejak Modul 4:

▸ **app/service/notification_rules.go**

```
package service

import (
    "errors"
    "fmt"

    "latihan-fiber/app/model"
)

var errUnknownEvent = errors.New("jenis event tidak dikenal")

// BuildMessage menyusun pesan dari sebuah event. Fungsi murni: tidak
// menyentuh database, Redis, maupun email, sehingga mudah diuji.
func BuildMessage(eventType string, e model.EnrollmentEvent, user model.User) (model.Message, error)
{
    course := e.CourseCode + " " + e.CourseName

    var subject, body string
    switch eventType {
    case model.EventEnrollmentCreated:
        subject = "KRS: berhasil terdaftar di " + course
        body = fmt.Sprintf("Halo %s, Anda tercatat sebagai peserta kelas %s.", user.Username, course)
    case model.EventEnrollmentDropped:
        subject = "KRS: keluar dari " + course
        body = fmt.Sprintf("Halo %s, Anda sudah tidak terdaftar di kelas %s.", user.Username, course)
    case model.EventEnrollmentSwapped:
        from := e.FromCourseCode + " " + e.FromCourseName
        subject = "KRS: pindah dari " + from + " ke " + course
        body = fmt.Sprintf("Halo %s, Anda telah pindah dari kelas %s ke kelas %s.", user.Username,
from, course)
```

```
    default:
        return model.Message{}, fmt.Errorf("%w: %q", errUnknownEvent, eventType)
    }

    return model.Message{To: user.Email, Subject: subject, Body: body}, nil
}
```

▸ **app/service/notification_worker.go**

```
package service

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "log/slog"

    "github.com/hibiken/asynq"

    "latihan-fiber/app/model"
    "latihan-fiber/app/repository"
    "latihan-fiber/helper"
)

// NotificationWorker mengerjakan task "notification:send".
//
// Bagi worker, Asynq adalah framework-nya — sama seperti Fiber bagi API.
// Karena itu method di sini menerima *asynq.Task, seperti method service
// lain menerima *fiber.Ctx.
type NotificationWorker struct {
    users         repository.UserRepository
    notifications repository.NotificationRepository
    tx            *repository.TxManager
    mailer        helper.Mailer
    logger        *slog.Logger
}

func NewNotificationWorker(
    users repository.UserRepository,
    notifications repository.NotificationRepository,
    tx *repository.TxManager,
    mailer helper.Mailer,
    logger *slog.Logger,
) *NotificationWorker {
    return &NotificationWorker{users: users, notifications: notifications,
        tx: tx, mailer: mailer, logger: logger}
}

// HandleSend mengembalikan error untuk meminta Asynq melakukan retry nanti,
// atau error yang membungkus asynq.SkipRetry untuk menyatakan "jangan di-retry,
// task ini tidak mungkin berhasil".
//
// Asynq menjamin setiap task dijalankan PALING SEDIKIT satu kali, bukan
// TEPAT satu kali. Task yang sama bisa datang dua kali, misalnya bila
// worker mati setelah mengirim email tetapi sebelum sempat melapor ke Redis.
// Karena itu handler ini harus idempoten: menjalankannya dua kali untuk
// task yang sama harus berakibat sama dengan menjalankannya sekali.
func (w *NotificationWorker) HandleSend(ctx context.Context, t asynq.Task) error {
    var job model.NotificationJob
    if err := json.Unmarshal(t.Payload(), &job); err != nil {
        return fmt.Errorf("%w: payload task rusak: %v", asynq.SkipRetry, err)
    }
    var event model.EnrollmentEvent
    if err := json.Unmarshal(job.Payload, &event); err != nil {
        return fmt.Errorf("%w: payload event %d rusak: %v", asynq.SkipRetry, job.OutboxID, err)
    }

    user, err := w.users.FindByID(ctx, event.UserID)
    if errors.Is(err, repository.ErrNotFound) {
        // Akunnya sudah dihapus. Melakukan retry tidak akan memunculkannya.
        return fmt.Errorf("%w: user %d tidak ditemukan", asynq.SkipRetry, event.UserID)
```

```
    }
    if err != nil {
        return err // database sedang bermasalah: layak di-retry
    }

    msg, err := BuildMessage(job.EventType, event, user)
    if err != nil {
        return fmt.Errorf("%w: %v", asynq.SkipRetry, err)
    }

    alreadySent := false
    err = w.tx.WithTxOnce(ctx, func(tx repository.DBTX) error {
        // Claim dan pengiriman berada di dalam SATU transaction:
        //   - pengiriman gagal  -> claim ikut dibatalkan -> attempt
        //     berikutnya masih dapat mengirim;
        //   - pengiriman sukses -> claim di-commit -> attempt berikutnya
        //     melihat claim ini dan tidak mengirim lagi.
        claimed, err := w.notifications.Claim(ctx, model.Notification{
            OutboxID: job.OutboxID, UserID: user.ID, Channel: "email",
            Subject: msg.Subject, Body: msg.Body,
        })
        if err != nil {
            return err
        }
        if !claimed {
            alreadySent = true
            return nil
        }
        return w.mailer.Send(ctx, msg)
    })

    if errors.Is(err, helper.ErrPermanentDelivery) {
        return fmt.Errorf("%v: %w", asynq.SkipRetry, err)
    }
    if err != nil {
        return err
    }

    retry, _ := asynq.GetRetryCount(ctx)
    w.logger.Info("notification_processed",
        slog.Int64("outbox_id", job.OutboxID),
        slog.String("event", job.EventType),
        slog.Int("retry", retry),
        slog.Bool("already_sent", alreadySent))
    return nil
}
```

Checkpoint kedua

```
go build ./app/... ./helper/...
go vet ./app/... ./helper/...
```

LANGKAH 9 — Konfigurasi Worker dan Titik Masuk Program

▸ **config/queue.go**

```
package config

import (
    "context"
    "fmt"
    "log/slog"
    "os"
    "time"

    "github.com/hibiken/asynq"

    "latihan-fiber/app/model"
)
```

```
// RedisOpt membaca alamat Redis dari .env. Objek yang sama dipakai oleh
// client (relay), server (worker), dan inspector (endpoint admin).
func RedisOpt() asynq.RedisClientOpt {
    return asynq.RedisClientOpt{
        Addr:     GetEnv("REDIS_ADDR", "localhost:6379"),
        Password: GetEnv("REDIS_PASSWORD", ""),
        DB:       GetEnvInt("REDIS_DB", 0),

        // Timeout dibuat pendek. Saat Redis mati, kita ingin segera
        // tahu, bukan menunggu lama dengan koneksi database yang ter-lock.
        DialTimeout:  2 * time.Second,
        ReadTimeout:  2 * time.Second,
        WriteTimeout: 2 * time.Second,
    }
}

// NewWorkerServer menyiapkan server Asynq — "Fiber"-nya worker.
func NewWorkerServer(logger *slog.Logger) *asynq.Server {
    return asynq.NewServer(RedisOpt(), asynq.Config{
        // Jumlah task yang dikerjakan bersamaan. Setiap task
        // notifikasi memegang satu koneksi PostgreSQL selama email dikirim,
        // sehingga angka ini harus lebih kecil dari DB_MAX_CONNS.
        Concurrency: GetEnvInt("WORKER_CONCURRENCY", 5),

        // Worker HANYA mengambil task dari queue yang disebut di sini.
        // Nama queue harus sama persis dengan yang dipakai relay.
        Queues: map[string]int{model.TaskSendNotification: 1},

        // Delay sebelum melakukan retry. n adalah jumlah retry yang
        // SUDAH terjadi, dimulai dari 0, sehingga delay-nya 2, 4, 6, 8, lalu
        // 10 detik. Bawaan Asynq jauh lebih panjang (belasan detik sampai
        // beberapa menit) — cocok untuk production, terlalu lama untuk praktikum.
        RetryDelayFunc: func(n int, err error, t *asynq.Task) time.Duration {
            return time.Duration((n+1)*2) * time.Second
        },

        // Saat worker dimatikan, task yang sedang berjalan diberi waktu
        // 10 detik untuk selesai. Yang belum selesai dikembalikan ke queue
        // dan akan dikerjakan ulang — alasan lain handler harus idempoten.
        ShutdownTimeout: 10 * time.Second,

        ErrorHandler: asynq.ErrorHandlerFunc(func(ctx context.Context, t *asynq.Task, err error) {
            retry, _ := asynq.GetRetryCount(ctx)
            maxRetry, _ := asynq.GetMaxRetry(ctx)
            logger.Warn("task_failed",
                slog.String("type", t.Type()),
                slog.String("attempt", fmt.Sprintf("%d/%d", retry+1, maxRetry+1)),
                slog.String("error", err.Error()))
        }),

        Logger:   newAsynqLogger(logger),
        LogLevel: asynq.WarnLevel,
    })
}

// asynqLogger menyalurkan log internal Asynq ke slog, agar seluruh log
// aplikasi tetap satu format dan satu tujuan seperti sejak Modul 3.
type asynqLogger struct{ l *slog.Logger }

func newAsynqLogger(l *slog.Logger) asynqLogger {
    return asynqLogger{l: l.With(slog.String("component", "asynq"))}
}

func (a asynqLogger) Debug(args ...any) { a.l.Debug(fmt.Sprint(args...)) }
func (a asynqLogger) Info(args ...any)  { a.l.Info(fmt.Sprint(args...)) }
func (a asynqLogger) Warn(args ...any)  { a.l.Warn(fmt.Sprint(args...)) }
func (a asynqLogger) Error(args ...any) { a.l.Error(fmt.Sprint(args...)) }

// Fatal wajib menghentikan proses: itulah kontrak interface Logger milik Asynq.
func (a asynqLogger) Fatal(args ...any) {
    a.l.Error(fmt.Sprint(args...))
```

```
    os.Exit(1)
}
```

Kode penyalaan worker ditulis di file terpisah, tetapi masih di package main. Karena itu mulai pertemuan ini program dijalankan dengan go run . (dengan titik) — bukan go run main.go, yang hanya mengompilasi satu file dan akan mengeluh fungsi startWorker tidak ditemukan.

▸ **worker.go**

```
package main

import (
    "context"
    "log/slog"
    "time"

    "github.com/hibiken/asynq"
    "github.com/jackc/pgx/v5/pgxpool"

    "latihan-fiber/app/model"
    "latihan-fiber/app/repository"
    "latihan-fiber/app/service"
    "latihan-fiber/config"
    "latihan-fiber/helper"
)

// startWorker menyalakan dua komponen background:
//
//  1. relay  : memindahkan event dari tabel outbox ke Redis;
//  2. server : mengambil task dari Redis lalu mengerjakannya.
//
// Mengembalikan fungsi untuk menghentikan keduanya, atau nil bila gagal.
func startWorker(ctx context.Context, logger *slog.Logger, pool *pgxpool.Pool) func() {
    mailer, err := helper.NewLogMailer(
        "logs/mail.log",
        time.Duration(config.GetEnvInt("MAILER_DELAY_MS", 1500))*time.Millisecond,
        config.GetEnvInt("MAILER_FAIL_EVERY", 0),
    )
    if err != nil {
        logger.Error("gagal menyiapkan mailer", slog.String("error", err.Error()))
        return nil
    }

    outboxRepository := repository.NewOutboxRepository(pool)
    txManager := repository.NewTxManager(pool)

    client := asynq.NewClient(config.RedisOpt())
    relay := service.NewOutboxRelay(
        outboxRepository, txManager, client, logger,
        time.Duration(config.GetEnvInt("RELAY_INTERVAL_MS", 1000))*time.Millisecond,
        config.GetEnvInt("RELAY_BATCH", 100),
    )

    worker := service.NewNotificationWorker(
        repository.NewUserRepository(pool),
        repository.NewNotificationRepository(pool),
        txManager, mailer, logger,
    )

    // ServeMux memetakan JENIS task ke handler-nya, seperti route
    // memetakan URL ke handler pada Fiber.
    mux := asynq.NewServeMux()
    mux.HandleFunc(model.TaskSendNotification, worker.HandleSend)

    srv := config.NewWorkerServer(logger)
    if err := srv.Start(mux); err != nil {
        logger.Error("gagal menyalakan worker", slog.String("error", err.Error()))
        _ = client.Close()
        return nil
    }
```

```
    relayDone := make(chan struct{})
    go func() {
        relay.Run(ctx)
        close(relayDone)
    }()
    logger.Info("worker berjalan")

    return func() {
        <-relayDone // relay berhenti sendiri karena ctx sudah dibatalkan
        srv.Shutdown()
        _ = client.Close()
    }
}
```

main.go ditulis ulang seluruhnya. Isi startAPI sama dengan main.go Modul 10, dipindahkan ke dalam fungsi tersendiri dan ditambah NotificationService:

▸ **main.go**

```
package main

import (
    "context"
    "flag"
    "log/slog"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/gofiber/fiber/v2"
    "github.com/hibiken/asynq"
    "github.com/jackc/pgx/v5/pgxpool"

    "latihan-fiber/app/repository"
    "latihan-fiber/app/service"
    "latihan-fiber/config"
    "latihan-fiber/database"
    "latihan-fiber/helper"
    "latihan-fiber/route"
)

const minSecretLength = 32

func main() {
    config.LoadEnv()
    logger := config.NewLogger()

    // Satu program, tiga cara menjalankan:
    //   go run . -role=api     hanya melayani HTTP
    //   go run . -role=worker  hanya relay outbox dan worker queue
    //   go run . -role=all     keduanya dalam satu proses
    // Di production, api dan worker dijalankan sebagai proses terpisah agar
    // masing-masing dapat diperbanyak sesuai bebannya.
    role := flag.String("role", config.GetEnv("APP_ROLE", "all"), "api | worker | all")
    flag.Parse()
    runAPI := *role == "api" || *role == "all"
    runWorker := *role == "worker" || *role == "all"
    if !runAPI && !runWorker {
        logger.Error("role tidak dikenal", slog.String("role", *role))
        os.Exit(1)
    }

    pool, err := database.NewPool(context.Background())
    if err != nil {
        logger.Error("gagal terhubung ke database", slog.String("error", err.Error()))
        os.Exit(1)
    }
    defer pool.Close()
```

```
    // ctx dibatalkan ketika Ctrl+C ditekan atau sistem mengirim SIGTERM.
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()

    var app *fiber.App
    if runAPI {
        app = startAPI(logger, pool)
        if app == nil {
            os.Exit(1)
        }
    }

    var stopWorker func()
    if runWorker {
        stopWorker = startWorker(ctx, logger, pool)
        if stopWorker == nil {
            os.Exit(1)
        }
    }

    logger.Info("aplikasi berjalan", slog.String("role", *role))
    <-ctx.Done()

    // Urutan berhenti: pintu masuk ditutup lebih dulu (HTTP), baru kemudian
    // pekerja di belakangnya.
    if app != nil {
        if err := app.ShutdownWithTimeout(10 * time.Second); err != nil {
            logger.Error("shutdown HTTP tidak rapi", slog.String("error", err.Error()))
        }
    }
    if stopWorker != nil {
        stopWorker()
    }
    logger.Info("aplikasi berhenti dengan rapi")
}

// startAPI menyiapkan seluruh kebutuhan HTTP. Mengembalikan nil bila gagal.
func startAPI(logger *slog.Logger, pool *pgxpool.Pool) *fiber.App {
    jwtSecret := config.GetEnv("JWT_SECRET", "")
    if len(jwtSecret) < minSecretLength {
        logger.Error("JWT_SECRET tidak diisi atau terlalu pendek",
            slog.Int("minimal_karakter", minSecretLength))
        return nil
    }

    mongoClient, err := database.NewMongoClient(context.Background())
    if err != nil {
        logger.Error("gagal terhubung ke MongoDB", slog.String("error", err.Error()))
        return nil
    }
    mongoDB := mongoClient.Database(config.GetEnv("MONGO_DATABASE", "praktikum_backend"))

    userRepository := repository.NewUserRepository(pool)
    tokenRepository := repository.NewTokenRepository(pool)
    roleRepository := repository.NewRoleRepository(pool)
    courseRepository := repository.NewCourseRepository(pool)
    outboxRepository := repository.NewOutboxRepository(pool)
    notificationRepository := repository.NewNotificationRepository(pool)
    txManager := repository.NewTxManager(pool)
    reviewRepository := repository.NewReviewRepository(mongoDB)

    if err := reviewRepository.EnsureIndexes(context.Background()); err != nil {
        logger.Error("gagal membuat index MongoDB", slog.String("error", err.Error()))
        return nil
    }

    rawPermissions, err := roleRepository.LoadPermissions(context.Background())
    if err != nil {
        logger.Error("gagal memuat permission", slog.String("error", err.Error()))
        return nil
    }
```

```
    permissions := helper.NewPermissionSet(rawPermissions)

    jwtManager := helper.NewJWTManager(
        jwtSecret,
        config.GetEnv("JWT_ISSUER", "praktikum-backend"),
        time.Duration(config.GetEnvInt("JWT_ACCESS_TTL_MINUTES", 15))*time.Minute,
    )

    // Inspector hanya MEMBACA keadaan queue untuk endpoint admin. Ia tidak
    // menghubungi Redis saat dibuat, sehingga API tetap dapat menyala
    // walaupun Redis sedang mati.
    inspector := asynq.NewInspector(config.RedisOpt())

    app := config.NewApp(logger, route.Dependencies{
        Pool:        pool,
        Mongo:       mongoClient,
        JWT:         jwtManager,
        Permissions: permissions,
        UserService: service.NewUserService(userRepository, permissions),
        AuthService: service.NewAuthService(
            userRepository, tokenRepository, jwtManager, permissions,
            time.Duration(config.GetEnvInt("JWT_REFRESH_TTL_DAYS", 7))*24*time.Hour,
        ),
        CourseService: service.NewCourseService(courseRepository, outboxRepository, txManager),
        ReviewService: service.NewReviewService(reviewRepository, courseRepository, permissions),
        NotificationService: service.NewNotificationService(
            notificationRepository, outboxRepository, inspector),
    })

    // Sumber daya milik API ditutup setelah server HTTP berhenti.
    app.Hooks().OnShutdown(func() error {
        ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
        defer cancel()
        _ = inspector.Close()
        return mongoClient.Disconnect(ctx)
    })

    port := config.GetEnv("APP_PORT", "3000")
    go func() {
        if err := app.Listen(":" + port); err != nil {
            logger.Error("server berhenti", slog.String("error", err.Error()))
            os.Exit(1)
        }
    }()
    logger.Info("HTTP berjalan", slog.String("port", port))
    return app
}
```

Checkpoint ketiga

```
go build ./...
go vet ./...
```

LANGKAH 10 — Endpoint Notifikasi dan Monitoring

▸ **app/service/notification_service.go**

```
package service

import (
    "time"

    "github.com/gofiber/fiber/v2"
    "github.com/hibiken/asynq"

    "latihan-fiber/app/model"
    "latihan-fiber/app/repository"
    "latihan-fiber/helper"
)
```

```
// QueueInspector adalah kemampuan membaca keadaan queue.
// *asynq.Inspector memenuhi interface ini.
type QueueInspector interface {
    GetQueueInfo(queue string) (*asynq.QueueInfo, error)
}

// NotificationService melayani endpoint HTTP seputar notifikasi.
type NotificationService struct {
    notifications repository.NotificationRepository
    outbox        repository.OutboxRepository
    inspector     QueueInspector
}

func NewNotificationService(
    notifications repository.NotificationRepository,
    outbox repository.OutboxRepository,
    inspector QueueInspector,
) *NotificationService {
    return &NotificationService{notifications: notifications, outbox: outbox, inspector: inspector}
}

// ---------- GET /notifications/mine ----------

func (s *NotificationService) Mine(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    current, ok := helper.CurrentUser(c)
    if !ok {
        return helper.Unauthorized("belum terautentikasi")
    }

    list, err := s.notifications.FindByUser(ctx, current.UserID, 20)
    if err != nil {
        return helper.Internal(err)
    }
    return helper.Success(c, fiber.StatusOK, "notifikasi berhasil diambil", list)
}

// ---------- GET /admin/queue ----------
// Dijaga middleware dengan permission queue:monitor.

func (s *NotificationService) Monitor(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    pending, published, oldest, err := s.outbox.Stats(ctx)
    if err != nil {
        return helper.Internal(err)
    }
    sent, err := s.notifications.Count(ctx)
    if err != nil {
        return helper.Internal(err)
    }

    stats := model.QueueStats{
        OutboxPending:     pending,
        OutboxPublished:   published,
        OldestPendingAge:  oldest.Round(time.Millisecond).String(),
        NotificationsSent: sent,
    }

    // Redis yang mati tidak boleh membuat endpoint ini gagal seluruhnya.
    // Justru saat Redis mati, admin paling membutuhkan angka outbox_pending
    // untuk melihat berapa notifikasi yang sedang tertahan.
    info, err := s.inspector.GetQueueInfo(model.QueueNotifications)
    if err != nil {
        stats.QueueError = "queue tidak dapat dibaca: Redis tidak dapat dihubungi atau queue belum
pernah dipakai"
    } else {
        stats.Queue = &model.QueueInfo{
```

```
            Name: info.Queue, Pending: info.Pending, Active: info.Active,
            Scheduled: info.Scheduled, Retry: info.Retry, Archived: info.Archived,
            Completed: info.Completed, Processed: info.Processed, Failed: info.Failed,
        }
    }
    return helper.Success(c, fiber.StatusOK, "keadaan queue berhasil diambil", stats)
}
```

Tambahkan satu field pada Dependencies di route/route.go:

▸ **route/route.go — Dependencies**

```
type Dependencies struct {
    Pool          *pgxpool.Pool
    Mongo         *mongo.Client
    JWT           *helper.JWTManager
    Permissions   *helper.PermissionSet
    UserService   *service.UserService
    AuthService   *service.AuthService
    CourseService *service.CourseService
    ReviewService *service.ReviewService

    NotificationService *service.NotificationService
}
```

Lalu tambahkan di akhir fungsi Register:

▸ **route/route.go — akhir Register**

```
    // --- notifikasi (diisi oleh worker, dibaca lewat API) ---
    notifications := api.Group("/notifications",
        middleware.RequireJSON,
        middleware.RequireAuth(deps.JWT))
    notifications.Get("/mine", deps.NotificationService.Mine)

    admin := api.Group("/admin",
        middleware.RequireJSON,
        middleware.RequireAuth(deps.JWT))
    admin.Get("/queue", middleware.RequirePermission(perms, "queue:monitor"),
deps.NotificationService.Monitor)
```

LANGKAH 11 — Menjalankan Dua Proses

Buka dua terminal di folder proyek:

```
# terminal 1
go run . -role=api

# terminal 2
go run . -role=worker
```

Simpan alat uji dari dosen sebagai tools/redeliver/main.go (dipakai di Bagian C). Lalu lakukan percobaan pertama dari terminal ketiga, memakai token mahasiswa:

```
curl -X POST http://localhost:3000/api/v1/courses/2/enroll \
  -H "Authorization: Bearer <token>" -H "Content-Type: application/json"

# tunggu 2-3 detik
curl http://localhost:3000/api/v1/notifications/mine \
  -H "Authorization: Bearer <token>"
```

Bila notifikasi muncul dan logs/mail.log bertambah satu baris, jalur dasarnya sudah tersambung. Itu baru permulaan: sebagian besar kesalahan di modul ini baru terlihat pada Bagian C.

Eksplorasi sendiri

? Jalankan dua proses worker sekaligus (terminal 2 dan terminal 3), lalu daftarkan beberapa mahasiswa. Jumlahkan angka published pada log relay_done di kedua terminal, lalu bandingkan dengan jumlah baris outbox. Bagian mana dari kode yang menjamin kedua angka itu sama?

? Perhatikan kolom id pada tabel outbox setelah mencoba mendaftar ke kelas yang sama dua kali. Ada nomor yang terlewat. Mengapa? Petunjuk: BIGSERIAL tidak ikut dibatalkan oleh rollback.

? Ubah MAILER_DELAY_MS menjadi 5000 dan WORKER_CONCURRENCY menjadi 1, lalu daftarkan lima mahasiswa. Berapa lama notifikasi terakhir tiba? Berapa lama response API? Apa yang Anda simpulkan tentang beban API dan beban worker?

? Hentikan worker dengan Ctrl+C tepat ketika sebuah email sedang dikirim (atur MAILER_DELAY_MS cukup besar). Apa yang tertulis di log, dan apakah notifikasinya tetap terkirim?

C. Spesifikasi Penerimaan

Pekerjaan Anda dinyatakan benar bila SELURUH uji di bawah ini lolos. Bagian ini dijamin benar: bila hasil Anda berbeda dari yang tertulis, kodenya yang perlu diperiksa, bukan spesifikasinya.

Persiapan uji

1. Daftarkan dua akun lewat POST /api/v1/auth/register: mhsuji dengan email mhsuji@student.unair.ac.id, dan salahalamat dengan email salah@kampus.invalid. Password bebas. Login keduanya dan simpan access token-nya. Siapkan juga token admin seperti pada Modul 6.
2. Simpan skrip reset11.sql di bawah ini di folder proyek. Jalankan sebelum C.1 dan sebelum C.3.
3. Sebelum setiap uji, kosongkan isi logs/mail.log (hapus isinya, bukan foldernya).

▸ **reset11.sql**

```
-- reset11.sql — mengembalikan keadaan akun uji Modul 11
--   psql -U postgres -d praktikum_backend -f reset11.sql
--
-- HANYA menyentuh dua akun uji: mhsuji dan salahalamat. Data mahasiswa
-- lain, kelas, dan seluruh tabel tetap utuh.

-- 1. Riwayat notifikasi dan event milik akun uji.
DELETE FROM notifications
WHERE user_id IN (SELECT id FROM users WHERE username IN ('mhsuji', 'salahalamat'));

DELETE FROM outbox
WHERE (payload->>'user_id')::INT IN (SELECT id FROM users WHERE username IN ('mhsuji',
'salahalamat'));

-- 2. KRS milik akun uji.
DELETE FROM enrollments
WHERE user_id IN (SELECT id FROM users WHERE username IN ('mhsuji', 'salahalamat'));

-- 3. Counter peserta dihitung ulang dari baris enrollments yang sebenarnya.
UPDATE courses c
SET enrolled_count = (SELECT COUNT(*) FROM enrollments e WHERE e.course_id = c.id);
```

Menghitung baris email untuk satu alamat:

```
# macOS / Linux
grep -c "mhsuji@" logs/mail.log

# Windows PowerShell
(Select-String -Path logs\mail.log -Pattern "mhsuji@").Count
```

Mematikan dan menyalakan Redis (dipakai pada C.2):

Sistem operasi	Mematikan	Menyalakan
Windows (Memurai)	memurai.exe --service-stop	memurai.exe --service-start
macOS (Homebrew)	brew services stop redis	brew services start redis
Linux (Ubuntu)	sudo systemctl stop redis-server	sudo systemctl start redis-server

C.1 Jalur normal

Worker berjalan dengan MAILER_FAIL_EVERY=0. Sebagai mhsuji, jalankan berurutan:

No	Permintaan	Status yang diharapkan
1	POST /courses/2/enroll	201
2	POST /courses/2/enroll (diulang)	409 ALREADY_ENROLLED
3	DELETE /courses/2/enroll	204
4	POST /courses/2/enroll	201
5	POST /enrollments/swap dengan {"from_course_id": 2, "to_course_id": 4}	200

Tunggu 5 detik, lalu periksa tiga hal:

- Tabel outbox untuk mhsuji berisi TEPAT empat baris, urut menurut id: enrollment.created, enrollment.dropped, enrollment.created, enrollment.swapped. Permintaan nomor 2 tidak boleh meninggalkan jejak apa pun.
- GET /notifications/mine mengembalikan empat notifikasi, dari yang terbaru: "KRS: pindah dari IF102 Basis Data Lanjut ke IF302 Keamanan Informasi", "KRS: berhasil terdaftar di IF102 Basis Data Lanjut", "KRS: keluar dari IF102 Basis Data Lanjut", "KRS: berhasil terdaftar di IF102 Basis Data Lanjut".
- logs/mail.log berisi tepat 4 baris untuk mhsuji@.

C.2 Redis mati, KRS tetap berjalan

1. Matikan Redis. Biarkan API dan worker tetap berjalan. Log worker akan dipenuhi peringatan relay_delayed — itu wajar.
2. Sebagai mhsuji, daftar ke kelas 5, 6, dan 7. Ketiganya harus 201, masing-masing selesai dalam hitungan milidetik.
3. Tunggu 5 detik. GET /admin/queue (token admin) harus menunjukkan outbox_pending: 3 dan berisi field queue_error.
4. Nyalakan Redis. Dalam 10 detik, outbox_pending harus kembali 0 dan GET /notifications/mine milik mhsuji berisi 7 notifikasi (4 dari C.1 ditambah 3).

C.3 Temporary failure

1. Ubah .env menjadi MAILER_FAIL_EVERY=3, lalu jalankan ulang worker. Jalankan reset11.sql dan kosongkan logs/mail.log.
2. Sebagai mhsuji, daftar ke kelas 1, 2, 3, 4, 5, dan 6. Keenamnya harus 201.
3. Tunggu 20 detik. Ketiga angka berikut harus SAMA-SAMA 6: jumlah baris notifications milik mhsuji, jumlah notifikasi pada GET /notifications/mine, dan jumlah baris logs/mail.log untuk mhsuji@.
4. Log worker memuat minimal dua baris task_failed dengan pesan "pengiriman gagal sementara". Tanpa baris-baris ini, uji ini belum membuktikan apa pun — artinya kegagalannya tidak pernah terjadi.

C.4 Permanent failure

1. Kembalikan MAILER_FAIL_EVERY=0 dan jalankan ulang worker. Catat nilai queue.archived pada GET /admin/queue.
2. Sebagai salahalamat, daftar ke kelas 2. Harus 201: kegagalan email tidak boleh memengaruhi KRS.
3. Tunggu 15 detik. queue.archived bertambah TEPAT 1, queue.retry bernilai 0, dan log worker memuat TEPAT SATU baris task_failed yang menyebut kampus.invalid, dengan attempt "1/6".

Catatan: Asynq menulis peringatan "Retry exhausted for task id=..." untuk setiap task yang masuk archived, termasuk yang masuk archived karena SkipRetry. Kalimat itu tidak berarti jatah attempt-nya benar-benar habis — yang menentukan adalah jumlah baris task_failed.

C.5 Idempotency

1. Matikan worker (Ctrl+C). Sebagai mhsuji, daftar ke kelas 8. Karena relay berada di proses worker, event ini tertahan di outbox.
2. Cari id event-nya: SELECT MAX(id) FROM outbox. Pastikan published_at-nya masih NULL.
3. Jalankan: go run ./tools/redeliver -outbox <id> -copies 5. Sekarang ada lima salinan task di Redis, dan relay akan menambahkan satu lagi begitu worker menyala.
4. Nyalakan worker. Tunggu 8 detik. SELECT COUNT(*) FROM notifications WHERE outbox_id = <id> harus 1, dan logs/mail.log hanya bertambah satu baris untuk kelas tersebut. Log worker memuat TEPAT ENAM baris notification_processed untuk outbox_id tersebut: satu dengan already_sent: false dan lima dengan true. Kurang dari enam berarti sebagian salinan tidak pernah dikerjakan, sehingga idempotency-nya belum teruji.

C.6 Graceful shutdown

Ubah MAILER_DELAY_MS menjadi 5000 dan jalankan ulang worker. Daftar ke kelas 9, lalu tekan Ctrl+C pada terminal worker sekitar dua detik setelah muncul log relay_done. Worker tidak boleh langsung berhenti: ia menunggu email selesai dikirim, menulis notification_processed, lalu aplikasi berhenti dengan rapi. Notifikasi untuk kelas 9 harus ada. Kembalikan MAILER_DELAY_MS menjadi 1500 setelahnya.

C.7 Log

Pesan	Proses	Field yang wajib ada
relay_done	worker	published

Pesan	Proses	Field yang wajib ada
relay_delayed	worker	published, error — hanya saat Redis bermasalah
notification_processed	worker	outbox_id, event, retry, already_sent
task_failed	worker	type, attempt ("x/6"), error
request_failed	api	TIDAK BOLEH muncul pada seluruh uji C.1 sampai C.6

C.8 Checklist ringkas

- go build ./... dan go vet ./... bersih tanpa peringatan.
- C.1: empat event, empat notifikasi, empat email — dengan subjek yang tepat.
- C.2: tiga KRS diterima saat Redis mati, tiga notifikasi tiba setelah Redis menyala.
- C.3: enam = enam = enam, dan kegagalannya benar-benar terjadi.
- C.4: satu attempt, langsung masuk archived.
- C.5: enam salinan task, satu email.
- C.6: Ctrl+C tidak memutus email yang sedang dikirim.

D. Tugas Mandiri

D.1 Berita Acara Perbaikan (wajib, bobot terbesar)

Untuk setiap kesalahan yang Anda temukan, isi satu baris tabel berikut. Sembilan kesalahan berarti sembilan baris.

No	Lokasi (file, fungsi)	Gejala yang Anda amati	Uji yang mengungkapnya	Akar masalah	Perbaikan
1					
…					

Kolom "Uji yang mengungkapnya" diisi nama uji dari Bagian C (atau "compiler"). Untuk kesalahan yang lolos compiler, sertakan bukti sebelum dan sesudah perbaikan: keluaran kueri, potongan log, atau isi response. Satu kalimat "sudah diperbaiki" tanpa bukti tidak dinilai.

D.2 Notifikasi Perubahan Kuota untuk Seluruh Peserta

Ketika admin mengubah kuota kelas lewat PATCH /courses/:id (Modul 9), seluruh peserta kelas itu harus menerima email "KRS: kuota <kode> <nama> berubah menjadi <kuota>". Ketentuan:

1. Event baru course.quota_changed dicatat ke outbox di dalam transaction yang sama dengan perubahan kuota. Perhatikan: UpdateQuota saat ini hanya satu pernyataan SQL tanpa WithTx. Begitu ada outbox, ia menjadi dua pernyataan — jelaskan mengapa hal itu memaksa Anda memakai transaction.
2. Satu event kini menghasilkan BANYAK notifikasi. UNIQUE (outbox_id) tidak lagi cocok. Tulis migration 009 yang mengganti idempotency key-nya, dan jelaskan pilihan Anda.

3. Tentukan sendiri apakah worker mengirim seluruh email dalam satu task, atau memecahnya menjadi satu task per mahasiswa. Apa pun pilihannya, bila email ke-7 dari 40 gagal, retry tidak boleh mengirim ulang email ke-1 sampai ke-6.
4. Buktikan dengan kelas yang memiliki minimal lima peserta, MAILER_FAIL_EVERY=3, dan alat redeliver.

D.3 Bukti Uji

Lampirkan keluaran C.2 sampai C.5 dari kode Anda yang sudah benar. Lalu lakukan satu percobaan pembanding: kembalikan SATU kesalahan pilihan Anda yang lolos compiler, jalankan uji yang mengungkapnya, dan tunjukkan bahwa uji itu memang gagal. Uji yang tidak pernah terbukti mampu gagal belum terbukti mampu menangkap apa pun.

D.4 Analisis Singkat

1. Seorang teman mengusulkan cara yang "lebih sederhana" dari outbox: tetap enqueue task ke Redis setelah commit, tetapi bila gagal, coba lagi tiga kali di dalam handler. Sebutkan dua skenario konkret di mana usulan ini tetap kehilangan notifikasi atau tetap membuat mahasiswa menunggu.
2. Redis dijalankan ulang tanpa persistence ketika 100 task masih pending. Outbox sudah menandai semuanya di-publish. Apa yang terjadi pada 100 notifikasi itu? Bagaimana Anda mendeteksinya dari data yang sudah ada di PostgreSQL, dan bagaimana memulihkannya tanpa mengirim email ganda?
3. Jelaskan dengan kata-kata Anda sendiri mengapa worker memakai WithTxOnce, sedangkan CourseService tetap memakai WithTx. Apa yang dapat terjadi bila keduanya ditukar?
4. Pada uji C.1, email "keluar dari" dapat tiba lebih dulu daripada email "berhasil terdaftar". Kapan hal itu menjadi masalah nyata bagi pengguna? Usulkan satu cara mengatasinya beserta harga yang harus dibayar.

Ketentuan Pengumpulan

Nama berkas	Tugas11_NIM.pdf
Isi laporan	Berita Acara Perbaikan (D.1) berisi sembilan baris beserta buktinya, migration dan potongan kode penting D.2, keluaran uji D.3 termasuk percobaan pembanding, dan jawaban analisis D.4
Kode sumber	Diunggah ke repository masing-masing; tautan dicantumkan pada halaman pertama laporan. Jangan mengunggah folder logs/ maupun file .env
Batas waktu	Sebelum praktikum pertemuan berikutnya dimulai

> **Yang dinilai adalah pembuktian**
> Pada sistem asinkron, kode yang benar dan kode yang salah berperilaku sama persis selama tidak ada yang gagal. Satu-satunya pembeda adalah uji yang sengaja membuat sesuatu gagal.
> Karena itu laporan dengan kode yang benar tetapi tanpa bukti C.2 sampai C.5 — terutama tanpa percobaan pembanding pada D.3 — akan dinilai rendah, seberapa pun rapi kodenya.

Catatan Penyusunan

Penyusunan modul ini dibantu oleh AI untuk pengembangan ide, penyusunan struktur materi, dan penulisan draf awal. Seluruh isi — termasuk kode program, skema database, dan angka hasil pengujian — telah dikurasi, dijalankan, dan diverifikasi ulang oleh dosen pengampu sebelum digunakan.

Rujukan

- Asynq — README dan dokumentasi package: https://github.com/hibiken/asynq dan https://pkg.go.dev/github.com/hibiken/asynq
- Richardson, C. Pattern: Transactional outbox. microservices.io: https://microservices.io/patterns/data/transactional-outbox.html
- PostgreSQL 16 Documentation — SELECT, The Locking Clause (FOR UPDATE, SKIP LOCKED): https://www.postgresql.org/docs/16/sql-select.html
- Redis Documentation — Install Redis: https://redis.io/docs/latest/operate/oss_and_stack/install/
- Redis — How to run Redis on Windows natively (Memurai): https://redis.io/tutorials/howtos/how-to-run-redis-on-windows-natively-with-memurai/
- Memurai Documentation — Installation dan Windows Service: https://docs.memurai.com/en/installation
- RFC 2606 — Reserved Top Level DNS Names (domain .invalid): https://www.rfc-editor.org/rfc/rfc2606